PRJ301: Java Web Application Development

Slot 6, 7, 8, 9: Introduction to JDBC

VinhVT9@fe.edu.vn

Spring 2026

FPT University Can Tho

Recall: Classwork 1

Task: Welcome App
Print welcome message using parameters `name` and `color` from URL.

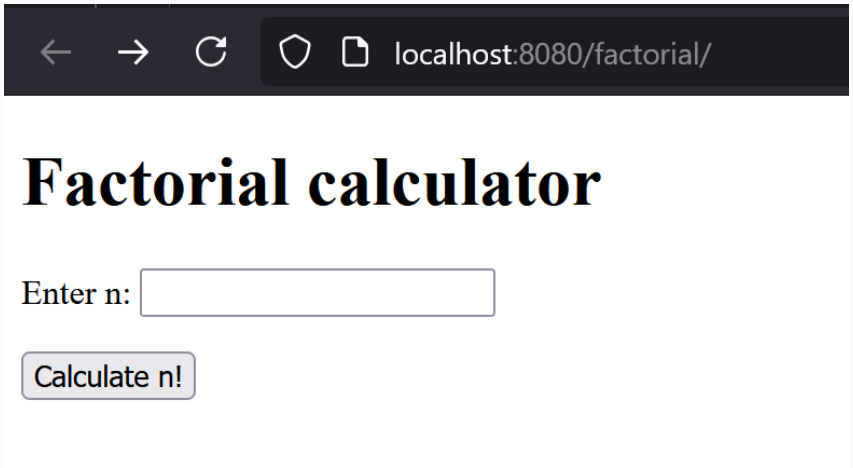
E.g.: http://localhost:8080/first-app/welcome?name=John&color=blue

FPT Education
FPT UNIVERSITY

Recall: Classwork 2

Task: Factorial Calculator
Calculate $n!$ for $1 < n < 21$. Handle invalid inputs.

- URL pattern: `/factorial`
- Constraint: $1 < n < 21$
- Input: integer n



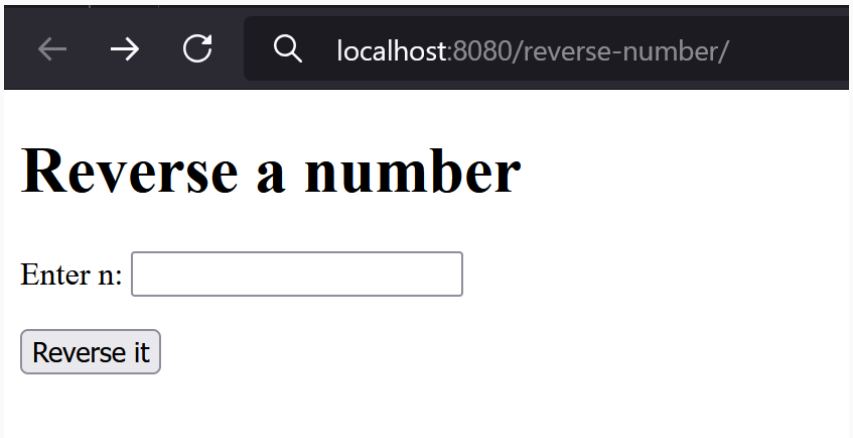
Legacy Reference UI

FPT Education
FPT UNIVERSITY

Recall: Classwork 3

Task: Number Reversal
Reverse a 3-digit number ($99 < n < 1000$).

- URL pattern: `/reverse-num`
- Input: integer n
- If entered n is not a 3-digit number, output an error.



Legacy Reference UI

FPT Education
FPT UNIVERSITY

Objectives

- **Separating View from Logic:**
  - Problems with writing HTML directly in Servlet.
  - Using JSP as the View layer.
  - Passing data with `RequestDispatcher`.
- **Understanding JDBC:**
  - What is JDBC? Architecture & Driver types.
  - Steps to connect and query a database.
- **Basic DB Operations:**
  - Connect to SQL Server.
  - Execute SELECT queries & process `ResultSet`.

FPT Education
FPT UNIVERSITY

Outline

From Servlet-only to Servlet + JSP

Introduction to JDBC

JDBC Driver Types

Steps to Develop a JDBC Application

FPT Education
FPT UNIVERSITY

From Servlet-only to Servlet + JSP

The Problem: HTML Inside Servlet

In the previous classworks, we wrote **all HTML output directly in the Servlet**:

```
protected void doGet(HttpServletRequest req,
                     HttpServletResponse res)
        throws ServletException, IOException {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    out.println("<html><body>");
    out.println("<h2>Result: " + result + "</h2>");
    out.println("</body></html>");
}
```

Why is this a problem?

- HTML is tangled with Java logic — hard to read and maintain.
- Designers cannot edit the UI without touching Java code.
- Adding CSS, tables, or complex layouts becomes unmanageable.

FPT Education
FPT UNIVERSITY

The Solution: Separate View with JSP

JavaServer Pages (JSP) lets us write HTML with embedded Java expressions — the proper **View layer**.

Servlet (Controller)

- Read request parameters.
- Execute business / DB logic.
- Store result as a **request attribute**.
- **Forward** to a JSP.

JSP (View)

- Receive the request attribute.
- Render **only HTML**.
- No JDBC, no business logic here.

This is the foundation of the **MVC pattern** we will use throughout the course.

FPT Education
FPT UNIVERSITY

Passing Data: RequestDispatcher

The bridge between Servlet and JSP is RequestDispatcher.

```
// --- In the Servlet ---
req.setAttribute("message", "Hello, " + name);
req.setAttribute("items", itemList); // any object

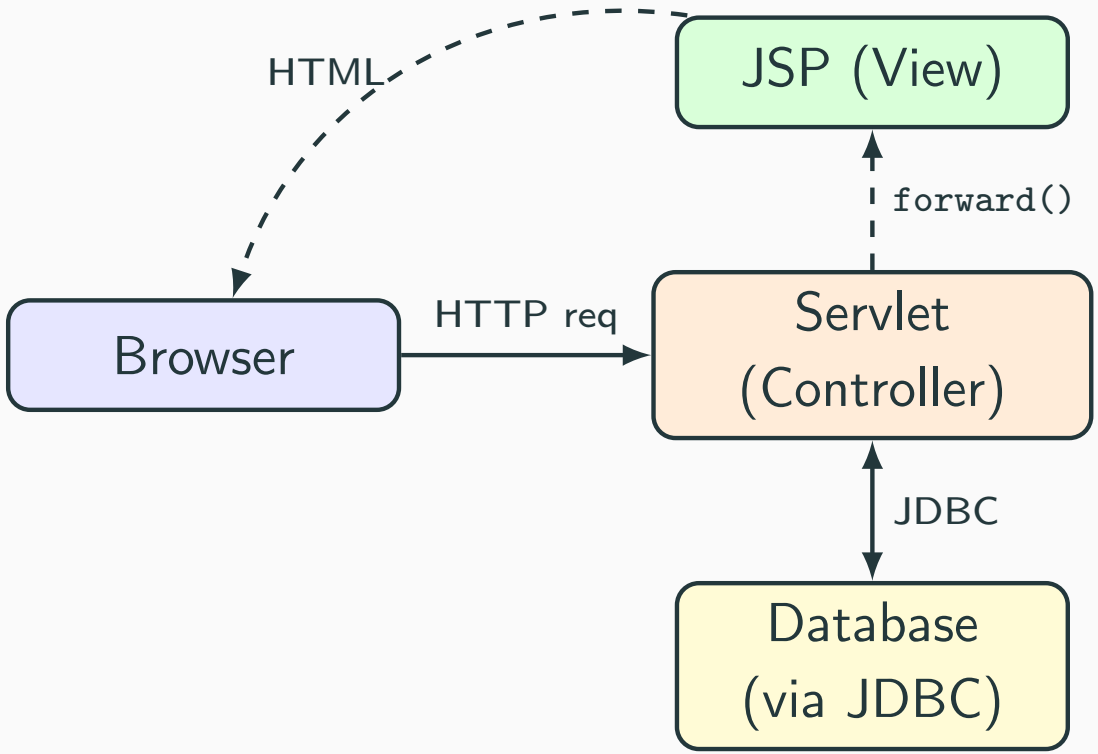
RequestDispatcher rd =
    req.getRequestDispatcher("result.jsp");
rd.forward(req, res);
```

In result.jsp (View)

- Read attribute: ${message} *(Expression Language)*
- Loop over list: <c:forEach var="item" items="${items}">
- Access field: ${item.name}

FPT Education
FPT UNIVERSITY

The Bigger Picture: Where Does JDBC Fit?



The Servlet **queries the DB via JDBC**, stores results as request attributes, then **forwards to JSP** to render the page. This slot covers the **JDBC** half.

FPT Education
FPT UNIVERSITY

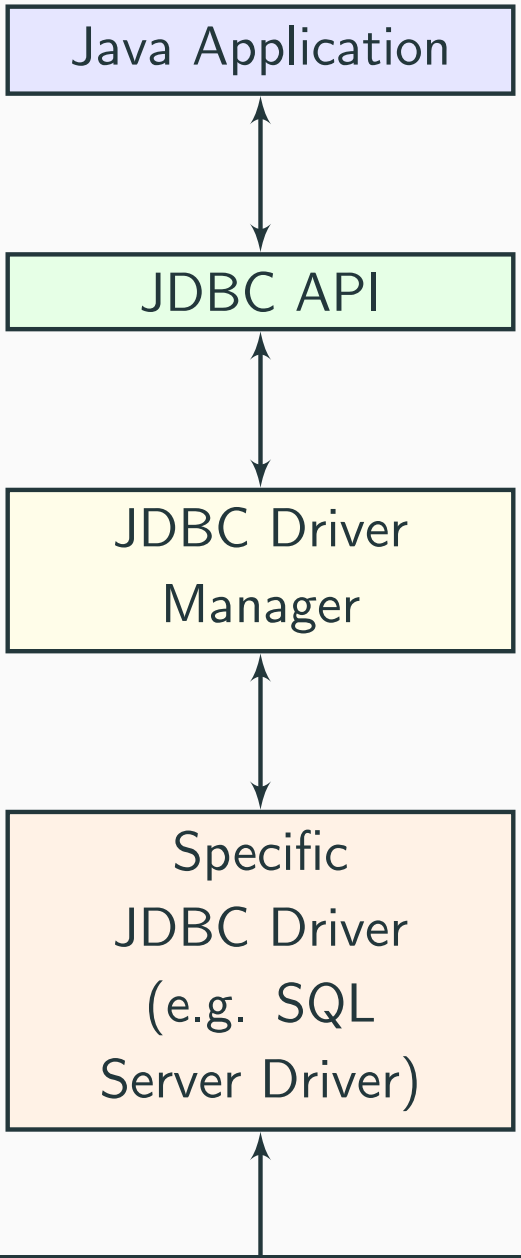
Introduction to JDBC

What is JDBC?

- **JDBC** stands for **Java Database Connectivity**.
- It is a Java API that can access any kind of tabular data, especially data stored in a **Relational Database**.
- JDBC makes everyday database tasks easy, allowing you to execute common SQL statements.
- It is a part of JavaSE (`java.sql` and `javax.sql` packages).

FPT Education
FPT UNIVERSITY

JDBC Architecture



FPT Education
FPT UNIVERSITY

JDBC Components

- **DriverManager**: Manages a list of database drivers. Matches connection requests from the java application with the proper database driver.
- **Driver**: The interface that handles the communications with the database server.
- **Connection**: Interface with all methods for contacting a database.
- **Statement**: Interface used to submit the SQL statements to the database.
- **ResultSet**: Objects which hold data retrieved from a database after you execute an SQL query using Statement objects.
- **SQLException**: This class handles any errors that occur in a database application.

FPT Education
FPT UNIVERSITY

JDBC Drivers

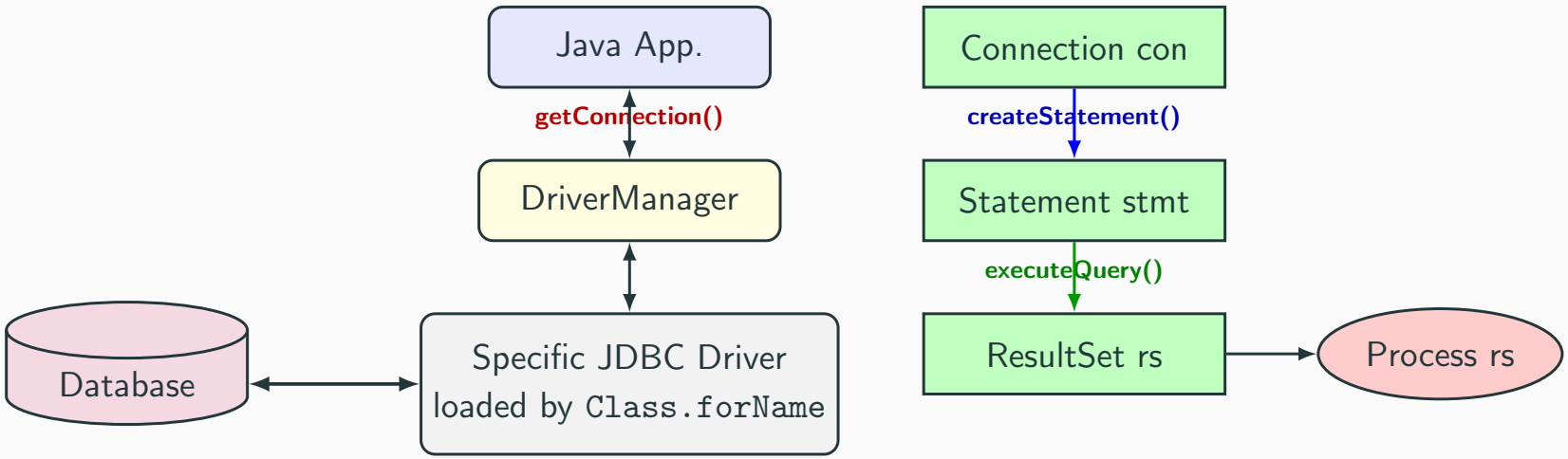
- A JDBC Driver is a software component enabling a Java application to interact with a database.
- Concrete driver classes are provided by specific vendors (Microsoft, Oracle, MySQL).
- **Dynamic Loading**: `Class.forName(DriverClass)` loads the concrete driver class dynamically.

Common Drivers

- SQL Server: `com.microsoft.sqlserver.jdbc.SQLServerDriver`
- MySQL: `com.mysql.cj.jdbc.Driver`
- Oracle: `oracle.jdbc.driver.OracleDriver`

FPT Education
FPT UNIVERSITY

Model of a JDBC Application



FPT Education
FPT UNIVERSITY

JDBC Driver Types

JDBC Driver Types: Comparison

Type	Name	Pure Java?	Middleware?	Performance	Use case
1	JDBC-ODBC Bridge	No (ODBC dep.)	No	Slow	Legacy Windows only
2	Native API	No (C/C++ lib)	No	Moderate	Proprietary client installs
3	Network Protocol	Yes	Yes	Moderate	Multi-DB via middleware
4	**Native Protocol**	**Yes**	**No**	**Best**	**Modern apps — use this**

Type 1 & 2 — avoid

- Type 1: platform-dependent (ODBC), removed from JDK 8+.
- Type 2: native C/C++ libraries must be installed on each machine.

Type 4 — recommended

Pure Java socket connection directly to the RDBMS.

Requires only a single JAR bundled in `WEB-INF/lib/`:

- SQL Server 2005+: `sqljdbc.jar`
- Open-source: `jtds.jar`

FPT Education FPT UNIVERSITY

Steps to Develop a JDBC Application

4+1 Steps to Develop a JDBC Application

1. **Load the JDBC Driver** `Class.forName(...)`
2. **Establish a DB Connection**
   `DriverManager.getConnection(...)`
3. **Create & Execute SQL Statements** `Statement / PreparedStatement`
4. **Process the Results** `ResultSet`
5. **Close the Connection** `rs / stmt / con .close()`

All JDBC classes live in `java.sql.*` (import required).

FPT Education
FPT UNIVERSITY

Step 1 & 2 – Load Driver & Connect (Type 4)

```
// Step 1: Load the JDBC driver (Type 4 - SQL Server)
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");

// Step 2: Establish a connection
String url = "jdbc:sqlserver://localhost:1433;"
           + "databaseName=Human;user=sa;password=1234;";
Connection con = DriverManager.getConnection(url);
```

Type 1 – DSN-based (ODBC)

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection con = DriverManager.getConnection("jdbc:odbc:myDSN");
```

FPT Education
FPT UNIVERSITY

Step 3 – Create & Execute SQL Statements

```java
// Statement - for static SQL
Statement stmt = con.createStatement();

// SELECT -> returns ResultSet
ResultSet rs = stmt.executeQuery("SELECT * FROM Employee");

// INSERT / UPDATE / DELETE -> returns row count
int rows = stmt.executeUpdate(
    "UPDATE Employee SET salary = 5000 WHERE empId = 1");

// PreparedStatement - for parameterised SQL (preferred)
PreparedStatement pStmt = con.prepareStatement(
    "INSERT INTO Employee (name, salary) VALUES (?, ?)");
pStmt.setString(1, "Alice");
pStmt.setDouble(2, 6000.0);
int rows2 = pStmt.executeUpdate();
```

FPT Education
FPT UNIVERSITY

Step 4 – Process the ResultSet

- A `ResultSet` maintains a **current row pointer**, initially at BOF.
- Traverse: `next()`, `previous()`, `first()`, `last()`.
- Read columns: `getXXX(columnIndex)` or `getXXX(columnLabel)`.

```
while (rs.next()) {
    int    id   = rs.getInt("empId");
    String name = rs.getString("name");
    double sal  = rs.getDouble("salary");
    System.out.println(id + " | " + name + " | " + sal);
}
```

Column label vs. name

`SELECT desc AS description FROM Employee`

→ use `rs.getString("description")` (label), not `"desc"` (alias).

FPT Education
FPT UNIVERSITY

Step 5 – Close the Connection

Opening order

1. `Connection`
2. `Statement`
3. `ResultSet`

Closing order (reverse)

1. `ResultSet`
2. `Statement`
3. `Connection`

One ResultSet per Connection

A single connection can only hold **one open ResultSet** at a time.

Solution: copy `rs1` data into an `ArrayList`, close `rs1`, then open `rs2`.

```
1 rs.close();
2 stmt.close();
3 con.close();
```

FPT Education
FPT UNIVERSITY

Summary

Servlet + JSP (MVC)

- Servlet = **Controller**: handles logic, calls JDBC.
- JSP = **View**: renders HTML only.
- `setAttribute()` + `RequestDispatcher.forward()` to pass data.

JDBC Driver Types

- **Type 1**: JDBC-ODBC Bridge — legacy, slow.
- **Type 4**: Native Protocol — pure Java, fastest, *use this*.

5 Steps to Use JDBC

1. `Class.forName(driver)`
2. `DriverManager.getConnection(u`
3. `con.createStatement()`
4. `stmt.executeQuery(sql)` → `ResultSet`
5. `rs / stmt / con .close()`

Key Rule

One `Connection` = one open `ResultSet` at a time. Transfer data to `ArrayList` before opening another.

FPT Education
FPT UNIVERSITY

Questions?

FPT Education
FPT UNIVERSITY